

Exercise-1—Design Patterns and Principles

Implementing the Singleton Pattern

Source code:

```
package test;
```

```
import computer.Computer;
```

```
public class ComputerBuilderTest {
```

```
    public static void main(String[] args) {
```

```
        System.out.println("\n[1] Basic Office PC");
```

```
        Computer officePc = new Computer.Builder("Intel Core i3-13100")
```

```
            .ram("8GB DDR4")
```

```
            .storage("256GB SSD")
```

```
            .motherboard("ASUS Prime H610M")
```

```
            .powerSupply("450W Bronze PSU")
```

```
            .wifi(true)
```

```
            .build();
```

```
        System.out.println(officePc);
```

```
        System.out.println("\n[2] High-End Gaming Rig");
```

```
        Computer gamingRig = new Computer.Builder("AMD Ryzen 9 7950X")
```

```
            .ram("64GB DDR5-6000")
```

```
            .storage("4TB NVMe Gen5 SSD")
```

```
            .gpu("NVIDIA RTX 4090 24GB")
```

```
            .motherboard("ASUS ROG Crosshair X670E")
```

```
            .powerSupply("1000W Platinum PSU")
```

```
.coolingSystem("360mm AIO Liquid Cooler")
.wifi(true)
.bluetooth(true)
.build();
System.out.println(gamingRig);
System.out.println("\n[3] Budget Home Build");
Computer budgetBuild = new Computer.Builder("Intel Core i5-12400")
    .ram("16GB DDR4")
    .storage("512GB SATA SSD")
    .powerSupply("550W Bronze PSU")
    .build();
System.out.println(budgetBuild);
System.out.println("\n[4] Professional Workstation");
Computer workstation = new Computer.Builder("Intel Xeon W-2295")
    .ram("128GB ECC DDR4")
    .storage("8TB NVMe RAID")
    .gpu("NVIDIA RTX A6000 48GB")
    .motherboard("ASUS Pro WS W480-ACE")
    .powerSupply("1200W Titanium PSU")
    .coolingSystem("Custom Water Loop")
    .wifi(false)
    .bluetooth(false)
    .build();
System.out.println(workstation);
System.out.println("\n[5] Minimal Server Node (CPU only)");
Computer server = new Computer.Builder("AMD EPYC 7763").build();
System.out.println(server);
```

```

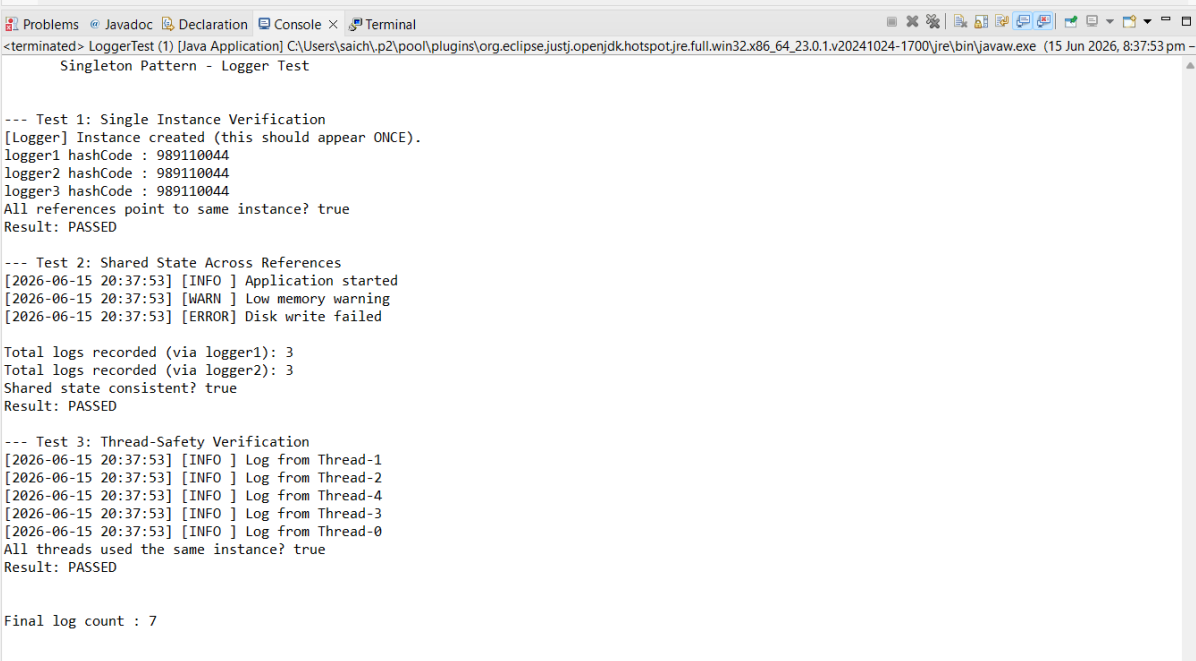
        System.out.println("\n[6] Testing validation — missing CPU:");

        try {
            Computer invalid = new Computer.Builder("").build();
        } catch (IllegalArgumentException e) {
            System.out.println(" Caught expected error: " + e.getMessage());
        }

        System.out.println("\nAll tests passed successfully.\n");
    }
}

```

OUTPUT:



```

<terminated> LoggerTest (1) [Java Application] C:\Users\saich\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin\javaw.exe (15 Jun 2026, 8:37:53 pm -
Singleton Pattern - Logger Test

--- Test 1: Single Instance Verification
[Logger] Instance created (this should appear ONCE).
logger1 hashCode : 989110044
logger2 hashCode : 989110044
logger3 hashCode : 989110044
All references point to same instance? true
Result: PASSED

--- Test 2: Shared State Across References
[2026-06-15 20:37:53] [INFO ] Application started
[2026-06-15 20:37:53] [WARN ] Low memory warning
[2026-06-15 20:37:53] [ERROR] Disk write failed

Total logs recorded (via logger1): 3
Total logs recorded (via logger2): 3
Shared state consistent? true
Result: PASSED

--- Test 3: Thread-Safety Verification
[2026-06-15 20:37:53] [INFO ] Log from Thread-1
[2026-06-15 20:37:53] [INFO ] Log from Thread-2
[2026-06-15 20:37:53] [INFO ] Log from Thread-4
[2026-06-15 20:37:53] [INFO ] Log from Thread-3
[2026-06-15 20:37:53] [INFO ] Log from Thread-0
All threads used the same instance? true
Result: PASSED

Final log count : 7

```